

Module 9: RxJS Essentials for Angular

Reactive Programming in Angular — mastering asynchronous data streams with Observables, Operators, Subjects, and Signals.

Module Overview

By the end of this module, you will be able to:

01

Reactive Programming

Understand the reactive programming model and why it matters in modern apps

02

Observables & Subscriptions

Create, subscribe to, and properly clean up Observable streams

03

RxJS Operators

Apply common operators like `map`, `filter`, `switchMap`, and `catchError`

04

Subjects & Signals

Distinguish Subject types, integrate Signals with RxJS, and choose the right tool



Why Reactive Programming?

Modern applications continuously receive data from multiple concurrent sources. Instead of repeatedly *polling* for updates, reactive programming lets your app **respond automatically** as new data arrives.

REST APIs & WebSockets

Seamlessly handle HTTP responses and real-time socket messages

User Events & Input

React to button clicks, keystrokes, and search input as streams

Notifications & Timers

Push alerts and scheduled events delivered reactively over time

What is RxJS?

RxJS (Reactive Extensions for JavaScript) is Angular's built-in library for composing asynchronous data streams with a rich operator ecosystem.

Angular Uses RxJS For

- → **HTTP Requests**
- → **Router Events**
- → **Form Value Changes**
- → **Timers & WebSockets**

Core Benefits

Reactive Model

Data flows downstream automatically

Rich Operators

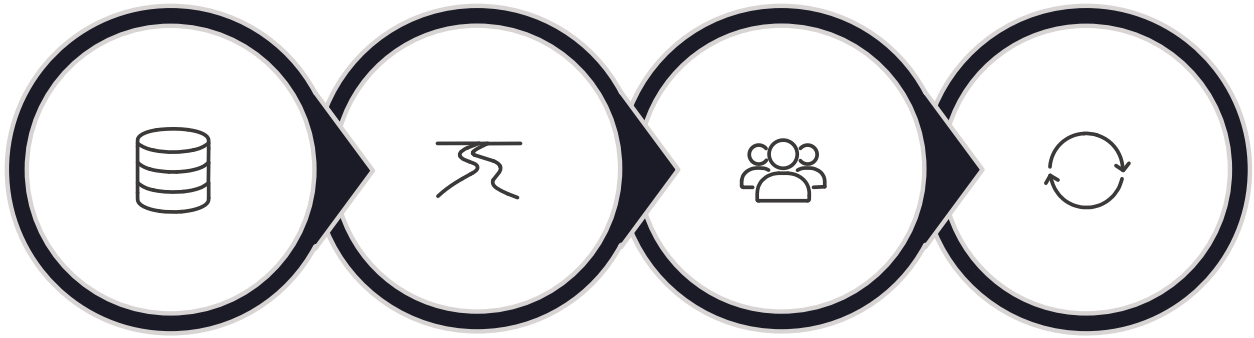
Transform, filter, and combine streams

Composable Code

Build clean, reusable data pipelines

Understanding Observables

An **Observable** represents a stream of values emitted over time — think of it as a live feed rather than a one-time response. Subscribers receive every emission until the stream completes or is unsubscribed.



Data Source **Observable** **Subscriber** **UI Update**

Common Observable Sources

- **HTTP APIs** — single or paginated responses
- **Form Controls** — every keystroke as a value
- **Button Clicks** — DOM events as a stream
- **Route Changes** — navigation events
- **WebSocket Messages** — real-time data

An Observable is **lazy** — it does nothing until something subscribes.

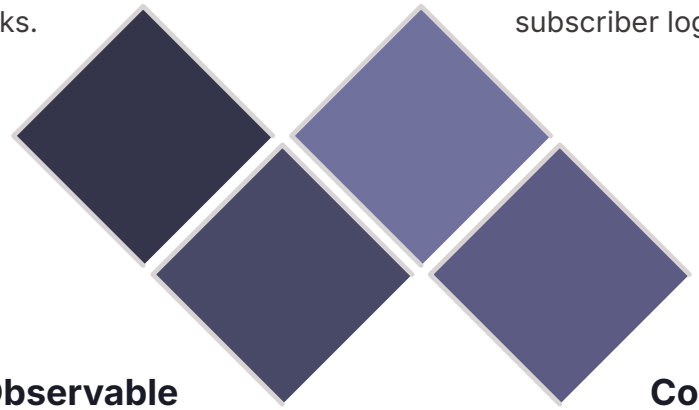
Subscription Lifecycle

Subscribe to Start

Begin receiving emissions and callbacks.

Receive Data

Handle each emission in subscriber logic.



Create Observable

Define data source to emit values.

Complete or Unsubscribe

Stop emissions and free resources.

Why Unsubscribe?

Active subscriptions keep memory allocated and callbacks running — even after a component is destroyed.

Best Practice

Use `takeUntilDestroyed()` to automatically clean up subscriptions when a component's lifecycle ends — no manual teardown required.

Prevent Memory Leaks

Release Resources

Boost Performance

Core RxJS Operators

Operators are pure functions that transform, filter, combine, or handle errors in Observable streams — chained together to build powerful, readable data pipelines.

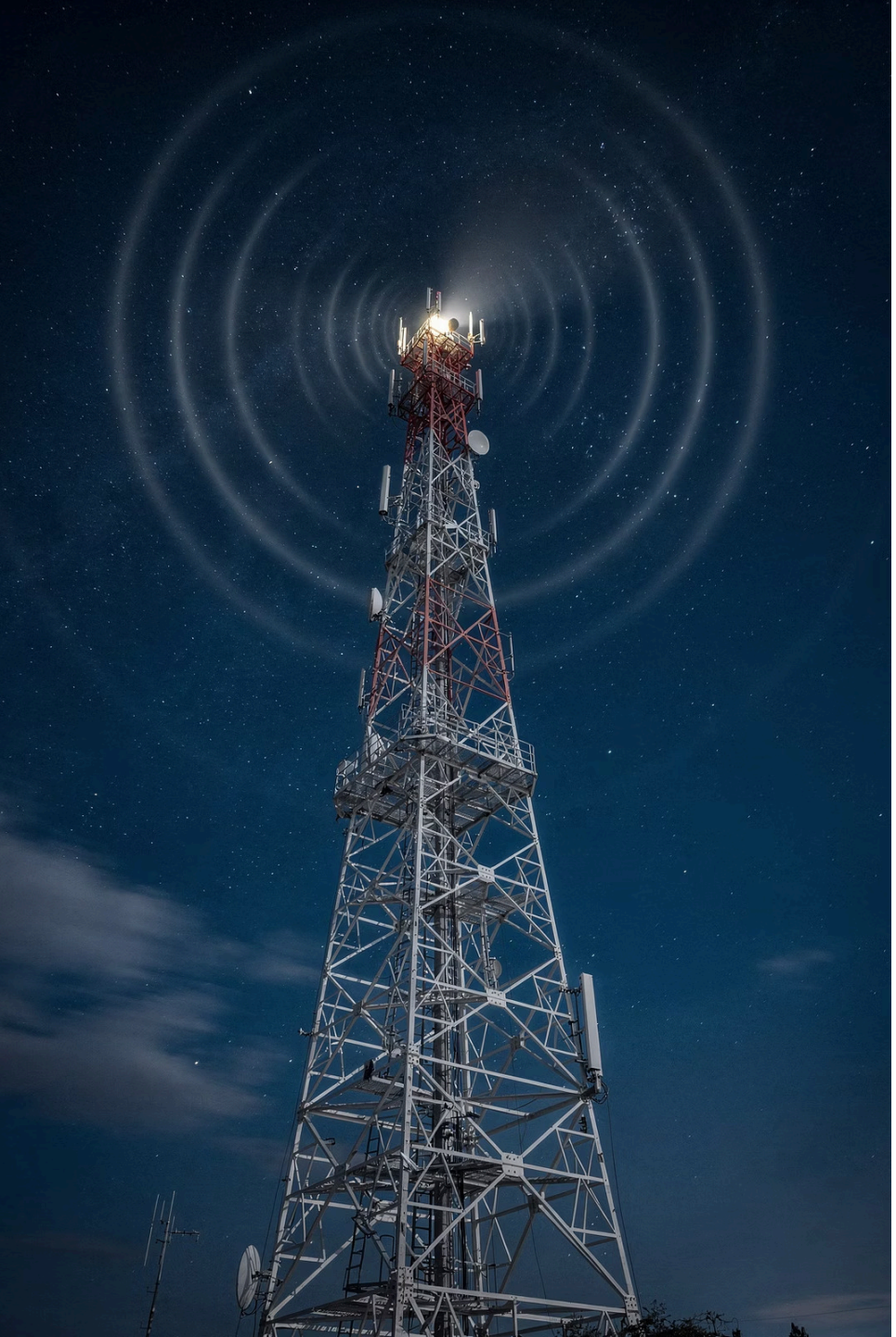
	`map` Transform each emitted value into a new shape
	`filter` Pass only values that meet a condition
	`tap` Inspect values for logging or side effects without modifying the stream
	`switchMap` Cancel previous inner Observable and switch to a new one
	`mergeMap` Process multiple inner Observables concurrently
	`catchError` Intercept errors and return a fallback Observable

Choosing the Right Operator

Knowing *when* to apply an operator matters more than memorizing every one. Use this reference to guide your decisions.

Scenario	Recommended Operator
Transform an API response	`map`
Filter out invalid data	`filter`
Logging or debugging a stream	`tap`
Search with request cancellation	`switchMap`
Parallel independent API calls	`mergeMap`
Graceful error handling	`catchError`

When in doubt between `switchMap` and `mergeMap`: if only the **latest** result matters (e.g. search), use `switchMap`. If all results matter equally, use `mergeMap`.



Subjects

A **Subject** acts as both an Observable *and* an Observer — it can emit values programmatically and multicast them to all active subscribers simultaneously.

Subject

Multicasts new values only. No memory of past emissions.

BehaviorSubject

Stores the latest value and replays it immediately to new subscribers.

ReplaySubject

Buffers and replays a configurable number of past emissions.

Use cases: shared application state, event broadcasting, dashboard updates, and push notifications.

Subject vs BehaviorSubject vs ReplaySubject

Type	Behavior
`Subject`	Emits only to **currently active** subscribers; late subscribers miss prior values
`BehaviorSubject`	Stores the **latest value**; new subscribers receive it immediately on subscribe
`ReplaySubject`	Replays a **configurable buffer** of past values to every new subscriber

When to Use Each

Subject → Events

Button clicks, one-off notifications

BehaviorSubject → Shared State

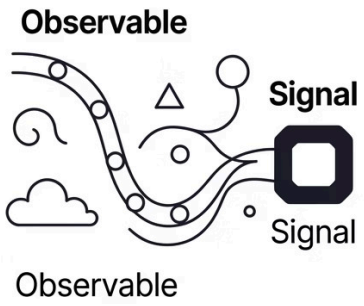
Auth status, user profile, app config

ReplaySubject → Cached Values

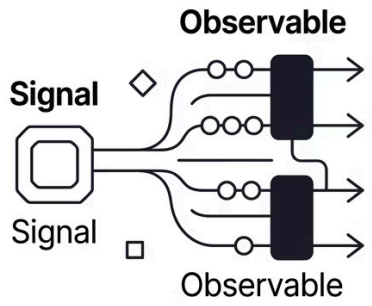
Message history, recent search results

Signals & RxJS Integration

Angular's **Signals** and **RxJS** are designed to coexist. Two simple APIs bridge the two worlds, so you can adopt Signals incrementally without rewriting existing code.



toSignal(): converts Observable to Signal for reactive UI state



toObservable(): converts Signal to Observable for existing RxJS pipelines

Common Scenarios

- Display HTTP results in the template using `toSignal()`
- Feed Signal state into an existing RxJS operator chain via `toObservable()`
- Reuse legacy RxJS services inside new Signal-based components
- Gradually modernize large codebases without a full rewrite

Signals vs RxJS

Both tools solve reactivity problems — but they excel in different contexts. Choose based on the nature of your data, not personal preference.

Dimension	Signals	RxJS
Primary Use	UI & component state	Async data streams
API Complexity	Simple, minimal API	Rich operator library
Dependency Tracking	Automatic	Manual composition
Learning Curve	Easy to learn	More advanced
Best For	Local UI reactivity	HTTP, WebSockets, router

Rule of thumb: Use **Signals** for state your UI reads. Use **RxJS** for anything that arrives asynchronously from outside the component.

Best Practices

- [ ] **Separate concerns**

  Use **Signals** for component/UI state; use **RxJS** for all async operations

- [ ] **Keep pipelines readable**

  Chain operators vertically with one concern per operator — avoid long, nested chains

- [ ] **Always handle errors**

  Use `catchError` in every HTTP pipeline to prevent unhandled stream termination

- [ ] **Clean up subscriptions**

  Leverage `takeUntilDestroyed()` or the `async` pipe — never leak subscriptions

- [ ] **Avoid nested subscriptions**

  Flatten with `switchMap` or `mergeMap` instead of subscribing inside a subscribe

Module Summary

What We Covered

01

Reactive Programming

02

Observables & Subscription Lifecycle

03

Core RxJS Operators

04

Subject Types

05

Signals & RxJS Integration

06

Best Practices

UP NEXT

Module 10: Routing & Navigation

- Angular Router setup
- Route Parameters & Guards
- Lazy Loading modules
- Navigation Strategies